

Database Design: PCC Student Grade Management System

This document showcases the database design for storing parsed data from bulk CSV files. It defines the schemas, entity relationships, data constraints, and DDL/DML queries used to manage student records, courses, assignments, and grades securely and efficiently.

CSV Data Schema & Ingestion Mapping

Bulk CSV ingestion parses records with a mix of profile, course, assignment, and performance data. The ingestion process normalizes this flat CSV structure into a relational database schema.

CSV Field Structure

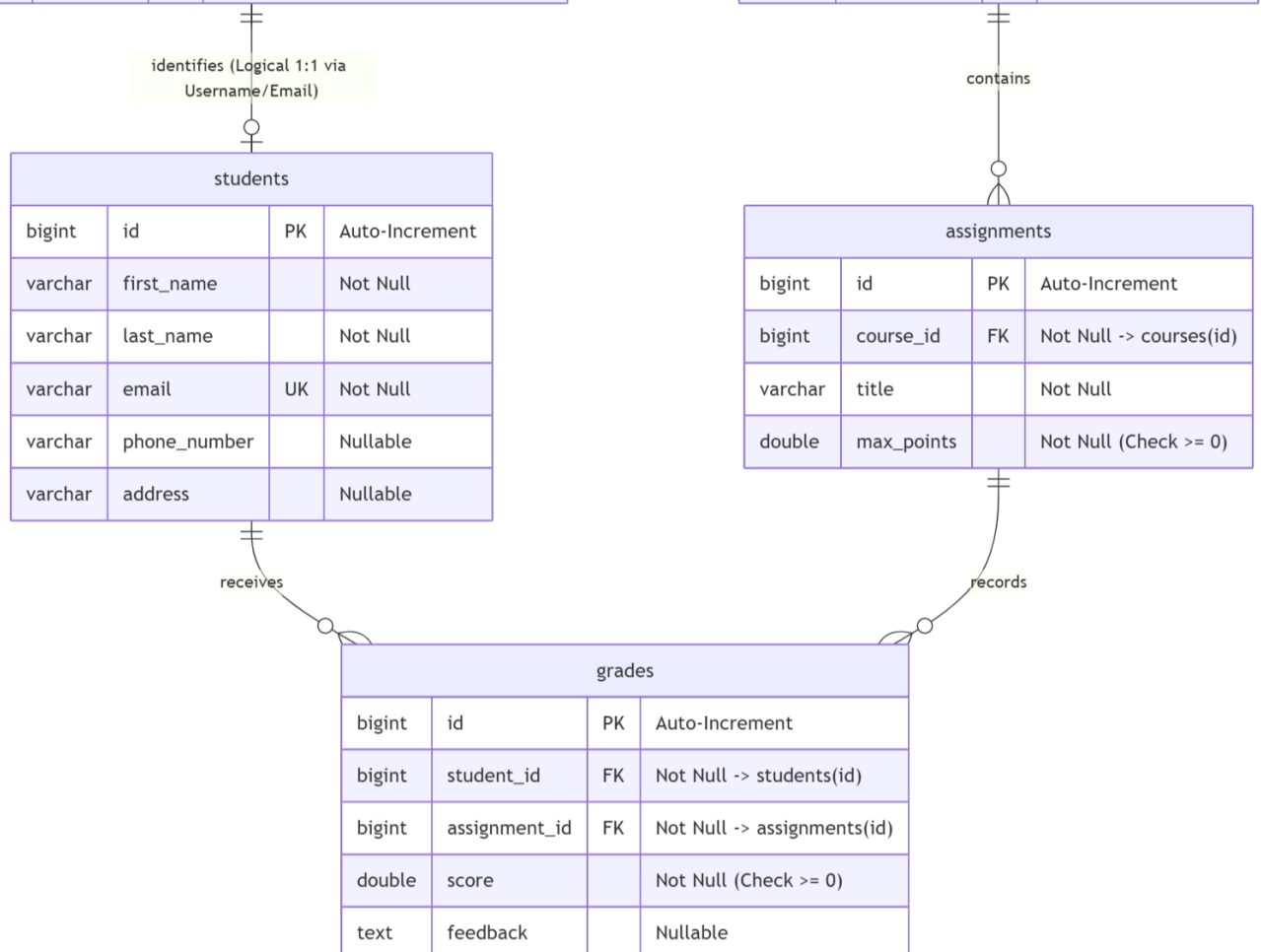
CSV Header Field	Required?	Target Table	Target Column	Data Type	Constraints / Validation
first_name	Yes	students	first_name	VARCHAR	Not Empty
last_name	Yes	students	last_name	VARCHAR	Not Empty
email	Yes	students users	email username	VARCHAR	Unique, Valid Email Format, Matches User Username
phone_number	No	students	phone_number	VARCHAR	Optional
address	No	students	address	VARCHAR	Optional
course_code	Yes	courses	course_code	VARCHAR	Unique Course Code (e.g., CIS101)
course_name	Yes	courses	course_name	VARCHAR	Not Empty
assignment_title	Yes	assignments	title	VARCHAR	Unique within Course scope
max_points	Yes	assignments	max_points	DOUBLE	Non-negative numeric value (≥ 0)
score	Yes	grades	score	DOUBLE	Non-negative numeric value (≥ 0)
feedback	No	grades	feedback	TEXT	Optional comments

Entity Relationship Diagram (ERD)

The database schema is designed in **Third Normal Form (3NF)** to avoid redundancy and ensure transactional integrity.

users			
bigint	id	PK	Auto-Increment
varchar	username	UK	Not Null (matches Student email)
varchar	password		Not Null (BCrypt encoded)
varchar	role		Not Null (ADMIN, TEACHER, STUDENT)

courses			
bigint	id	PK	Auto-Increment
varchar	course_code	UK	Not Null (e.g. CIS101)
varchar	course_name		Not Null
text	description		Nullable



Note

Authentication vs. Profile Separation: The `users` table handles authentication credentials (hashed passwords, security roles) while `students` handles personal contact info. They are decoupled and linked logically where the student's `email` corresponds to the user's `username`.

🔧 Data Definition Language (DDL)

The following DDL scripts define the schema with appropriate constraints, types, and indexes optimized for lookups.

```

-- 1. Create Users Table
CREATE TABLE users (
    id BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    username VARCHAR(255) UNIQUE NOT NULL,
    password VARCHAR(255) NOT NULL,
    role VARCHAR(50) NOT NULL,
    CONSTRAINT chk_user_role CHECK (role IN ('ADMIN', 'TEACHER', 'STUDENT'))
);

-- 2. Create Students Table
CREATE TABLE students (
    id BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    first_name VARCHAR(255) NOT NULL,
    last_name VARCHAR(255) NOT NULL,
    email VARCHAR(255) UNIQUE NOT NULL,
    phone_number VARCHAR(50),
    address VARCHAR(255)
);

-- 3. Create Courses Table
CREATE TABLE courses (
    id BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    course_code VARCHAR(100) UNIQUE NOT NULL,
    course_name VARCHAR(255) NOT NULL,
    description TEXT
);

-- 4. Create Assignments Table
CREATE TABLE assignments (
    id BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    course_id BIGINT NOT NULL,
    title VARCHAR(255) NOT NULL,
    max_points DOUBLE PRECISION NOT NULL,
    CONSTRAINT fk_assignment_course FOREIGN KEY (course_id) REFERENCES courses (id) ON DELETE CASCADE,
    CONSTRAINT uq_course_assignment_title UNIQUE (course_id, title),
    CONSTRAINT chk_assignment_max_points CHECK (max_points >= 0.0)
);

-- 5. Create Grades Table
CREATE TABLE grades (
    id BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    student_id BIGINT NOT NULL,
    assignment_id BIGINT NOT NULL,
    score DOUBLE PRECISION NOT NULL,
    feedback TEXT,
    CONSTRAINT fk_grade_student FOREIGN KEY (student_id) REFERENCES students (id) ON DELETE CASCADE,
    CONSTRAINT fk_grade_assignment FOREIGN KEY (assignment_id) REFERENCES assignments (id) ON DELETE CASCADE,
    CONSTRAINT uq_student_assignment_grade UNIQUE (student_id, assignment_id),
    CONSTRAINT chk_grade_score CHECK (score >= 0.0)
);

-- 6. Performance Optimization Indexes
CREATE INDEX idx_students_email ON students(email);
CREATE INDEX idx_courses_code ON courses(course_code);
CREATE INDEX idx_assignments_lookup ON assignments(course_id, title);
CREATE INDEX idx_grades_student_assignment ON grades(student_id, assignment_id);

```

Data Constraints Implemented:

1. **Primary Keys:** System-generated auto-incrementing IDs (`BIGINT GENERATED BY DEFAULT AS IDENTITY`) ensure robust indexing and avoid exposing domain IDs.

2. Unique Constraints:

- `users.username` & `students.email` : Prevent duplicate accounts.
- `courses.course_code` : Enforces code uniqueness (e.g. `CS101` cannot be duplicated).
- `uq_course_assignment_title` : An assignment name must be unique *within* a single course.
- `uq_student_assignment_grade` : A student can have only *one* grade entry per assignment.

3. Foreign Keys (Referential Integrity):

- Cascading deletions (`ON DELETE CASCADE`) are active on `assignments` and `grades` . If a student or course is deleted, corresponding grades/assignments are pruned to prevent orphaned data.

4. Value Range Restrictions (`CHECK`):

- `score >= 0.0` and `max_points >= 0.0` prevent entering negative scores.
- Roles are constrained strictly to `ADMIN` , `TEACHER` , or `STUDENT` .

Data Manipulation Language (DML) for CSV Ingestion

During CSV ingestion, the system reads each line and performs transactional database operations. Below is the SQL representation of the operations executed by the backend service.

Phase 1: Ingestion Pipelines

1. Ingest Student & User Account

If a student does not exist, a default student credentials account is created, followed by the student profile:

```
-- Check if Student profile exists by email
SELECT id FROM students WHERE email = 'jane.doe@pasadena.edu';

-- If not exists:
-- Step 1.1: Create associated security User
INSERT INTO users (username, password, role)
VALUES ('jane.doe@pasadena.edu', '$2a$10$eFDH52...', 'STUDENT');

-- Step 1.2: Insert Student profile details
INSERT INTO students (first_name, last_name, email, phone_number, address)
VALUES ('Jane', 'Doe', 'jane.doe@pasadena.edu', '626-555-0199', '1570 E Colorado Blvd, Pasadena, CA');

-- If student exists, update optional contact details (if provided in CSV):
UPDATE students
SET phone_number = '626-555-0199',
    address = '1570 E Colorado Blvd, Pasadena, CA'
WHERE email = 'jane.doe@pasadena.edu';
```

2. Ingest Course

```
-- Check if Course exists by course_code
SELECT id FROM courses WHERE course_code = 'CS101';

-- If not exists, insert new course:
INSERT INTO courses (course_code, course_name, description)
VALUES ('CS101', 'Introduction to Computer Science', 'Imported course Introduction to Computer Science');
```

3. Ingest Assignment

```
-- Check if Assignment exists for the course
SELECT id FROM assignments WHERE course_id = 1 AND title = 'Final Exam';

-- If not exists, insert new assignment:
INSERT INTO assignments (course_id, title, max_points)
VALUES (1, 'Final Exam', 100.0);
```

4. Ingest Grade (With Upsert Logic)

If the student already has a grade registered for the assignment, the record is updated; otherwise, a new one is saved. In SQL, this is equivalent to an **UPSERT (ON CONFLICT)**:

```
-- Upsert Grade record
INSERT INTO grades (student_id, assignment_id, score, feedback)
VALUES (5, 12, 92.5, 'Excellent performance on the final exam!')
ON CONFLICT (student_id, assignment_id)
DO UPDATE SET
    score = EXCLUDED.score,
    feedback = COALESCE(EXCLUDED.feedback, grades.feedback);
```



Diagnostic Queries

These DML queries provide analytics and power the backend data visualization features.

1. Retrieve a Student Scorecard

Fetches all grades, assignment metadata, and scores for a specific student profile.

```
SELECT
    c.course_code,
    c.course_name,
    a.title AS assignment_title,
    g.score,
    a.max_points,
    ROUND((g.score / a.max_points) * 100, 2) AS percentage,
    g.feedback
FROM grades g
JOIN assignments a ON g.assignment_id = a.id
JOIN courses c ON a.course_id = c.id
JOIN students s ON g.student_id = s.id
WHERE s.email = 'student@pasadena.edu'
ORDER BY c.course_code, a.title;
```

2. Calculate Assignment Summary Statistics

Computes class average, maximum score, minimum score, and submission count for a given assignment.

```
SELECT
    a.title AS assignment_title,
    c.course_code,
    COUNT(g.id) AS total_submissions,
    MIN(g.score) AS lowest_score,
    MAX(g.score) AS highest_score,
    ROUND(AVG(g.score), 2) AS average_score,
    a.max_points
```

```

a.max_points,
ROUND((AVG(g.score) / a.max_points) * 100, 2) AS average_percentage
FROM grades g
JOIN assignments a ON g.assignment_id = a.id
JOIN courses c ON a.course_id = c.id
WHERE a.id = 12
GROUP BY a.title, c.course_code, a.max_points;

```

3. Compute Course Percentiles

Computes rank profiles for students in a course based on their cumulative performance percentage.

```

WITH student_course_totals AS (
  SELECT
    s.id AS student_id,
    s.first_name || ' ' || s.last_name AS student_name,
    c.course_code,
    SUM(g.score) AS total_earned,
    SUM(a.max_points) AS total_possible,
    (SUM(g.score) / SUM(a.max_points)) * 100 AS cumulative_percentage
  FROM grades g
  JOIN assignments a ON g.assignment_id = a.id
  JOIN courses c ON a.course_id = c.id
  JOIN students s ON g.student_id = s.id
  GROUP BY s.id, s.first_name, s.lastName, c.course_code
)
SELECT
  student_name,
  course_code,
  ROUND(cumulative_percentage, 2) AS grade_percentage,
  ROUND(PERCENT_RANK() OVER (PARTITION BY course_code ORDER BY cumulative_percentage) * 100, 1) AS percentile
FROM student_course_totals
ORDER BY course_code, grade_percentage DESC;

```